

UNIVERSIDAD NACIONAL DE INGENIERÍA

Maestría en Inteligencia Artificial

Examen Parcial — Curso de Big Data

Análisis Exploratorio de Datos y Arquitectura Escalable de Big Data

Dataset: **ARM sgpmetE32.b1 (2021)**, estación meteorológica Southern Great Plains

Autor: Niels Pacheco-Barrios

Modalidad: Individual

Fecha: 7 de junio de 2026

Profesora: Mg. Rosa Virginia Encinas Quille

Dataset: github.com/encinasquille/DataLifecyclePy/raw/main/data/raw/ARM_sgpmetE32.b1.2021.nc

Fuente original: ARM Data Discovery, adc.arm.gov/discovery/#/results/s::sgpmetE32

1. Análisis Exploratorio de Datos (EDA)

Trabajé el dataset siguiendo el ciclo de vida de los datos que vi en clase. Partí de los datos crudos, hice el análisis exploratorio para encontrar los problemas del dataset (datos vacíos, duplicados, correlaciones, outliers y ruido), y a partir de ahí limpié y transformé los datos. En cada fase comento los desafíos que aparecen cuando este flujo se lleva a escala Big Data. Todo el análisis lo hice en Python con `xarray`, `pandas`, `matplotlib` y `seaborn`; el código está en el Anexo A y es reproducible.

1.1 Tipo de datos y origen

El dataset viene del programa **ARM (Atmospheric Radiation Measurement)** del Departamento de Energía de EE.UU. (Oak Ridge National Laboratory), el mismo caso de arquitectura Big Data en producción que vi en clase. Corresponde al instrumento **MET (Surface Meteorological Instrumentation)** de la instalación **E32** del observatorio **Southern Great Plains (SGP)**, en Medford, Oklahoma (lat 36.819°N, lon 97.820°W, alt 328 m s.n.m.).

Característica	Descripción
Tipo de datos	Serie temporal multivariada de sensores. Datos estructurados: medidas numéricas continuas más flags de calidad discretos. Es instrumentación científica de tipo IoT.
Origen	Sensores físicos (termohigrómetro HMP155, barómetro, anemómetro, pluviómetro de balancín TBRG) que escriben a un datalogger. El proceso de ingesta es <code>met_ingest -s sgp -f E32</code> (versión <code>ingest-met-4.42</code>).
Formato	NetCDF (Network Common Data Form), un formato binario autodescriptivo que es el estándar de la comunidad atmosférica. Da soporte a la I (interoperabilidad) de los principios FAIR.
Nivel de procesamiento	b1 : datos calibrados que ya pasaron el control de calidad automático del ARM. Los niveles que vi en clase son 00 (brutos), a1 (calibrados) y b1 (con QC aplicado).
Frecuencia	Promedios cada 60 segundos (resolución de 1 minuto), año completo 2021.
Identificación FAIR	Datastream <code>sgpmetE32.b1</code> , accesible desde el portal ARM Data Discovery, con metadatos ricos y documentación del instrumento (principios F indable, A ccessible, I nteroperable, R eusable).

1.2 Estructura del dataset: metadatos, registros y variables

El archivo NetCDF tiene **521,200 registros** (la dimensión única es `time`) y **31 variables de datos**, y ocupa **70.9 MB** en disco. El rango va del **2021-01-01 00:00** al **2021-12-31 23:59 UTC**. Un año completo a resolución de 1 minuto tendría 525,600 registros, así que la **completitud temporal es del 99.16%**: faltan 4,400 minutos por interrupciones del instrumento. La verificación de integridad confirmó **0 timestamps duplicados** y un índice temporal estrictamente creciente.

Las variables se agrupan en tres bloques según su tipo de dato (el desglose suma las 31 variables de datos, además de la coordenada `time`):

Grupo	Variables	Tipo	Ejemplos
Temporales (como variables de datos)	2	<code>datetime64[ns]</code>	<code>base_time</code> , <code>time_offset</code> (más la coordenada <code>time</code>)
Medidas físicas (cuantitativas continuas)	18	<code>float32</code>	<code>temp_mean</code> (°C), <code>rh_mean</code> (%), <code>atmos_pressure</code> (kPa), <code>wspd_arith_mean</code> (m/s), <code>tbrg_precip_total_corr</code> (mm), <code>lat/lon/alt</code>
Flags de control de calidad (categóricas codificadas)	11	<code>int32</code>	<code>qc_temp_mean</code> , <code>qc_rh_mean</code> , ... (bits empaquetados: bit 1 = valor faltante, bit 2 = < mínimo válido, bit 3 = > máximo válido, bit 4 = salto > <code>valid_delta</code>)

Los metadatos globales del archivo son la base de la gobernanza de datos que vi en clase. Documentan el sitio (`site_id: sgp`, `facility_id: E32`), el datastream (`sgpmetE32.b1`), la versión del software de ingesta, el intervalo de promediado (60 s), las ecuaciones de corrección del pluviómetro y la semántica de cada bit de calidad. Gracias a esto el archivo se entiende por sí solo, sin documentación externa, y se puede reutilizar.

Estadística descriptiva de las variables principales (datos crudos)

Variable	n	Media	Desv. est.	Mín	P5	P50	P95	Máx	Asimetría
Temperatura (°C)	521,199	15.05	11.51	-27.14	-3.72	16.16	33.13	39.91	-0.31
Humedad relativa (%)	521,196	70.20	22.04	14.22	30.74	73.67	99.70	101.50	-0.43
Presión atmosférica (kPa)	521,200	97.65	0.66	95.52	96.51	97.64	98.83	99.56	0.02
Presión de vapor (kPa)	521,200	1.36	0.82	0.00	0.33	1.12	2.76	3.63	0.40
Velocidad del viento (m/s)	521,200	4.70	2.67	0.00	1.26	4.09	9.90	22.95	1.04
Dirección del viento (°)	521,200	157.7	95.98	0.0	12.1	153.2	340.9	360.0	0.36
Precipitación corr. (mm)	521,200	0.00	0.02	0.00	0.00	0.00	0.00	2.11	30.51
Voltaje datalogger (V)	521,200	13.09	0.27	12.19	12.71	13.05	13.57	14.18	0.59

La precipitación es el total acumulado en cada intervalo de 1 minuto (unidad `mm` del archivo). La dirección del viento (`wdir_vec_mean`) ya viene como media vectorial calculada por el ARM, que maneja correctamente su naturaleza circular; sus estadísticos de la tabla son solo descriptivos.

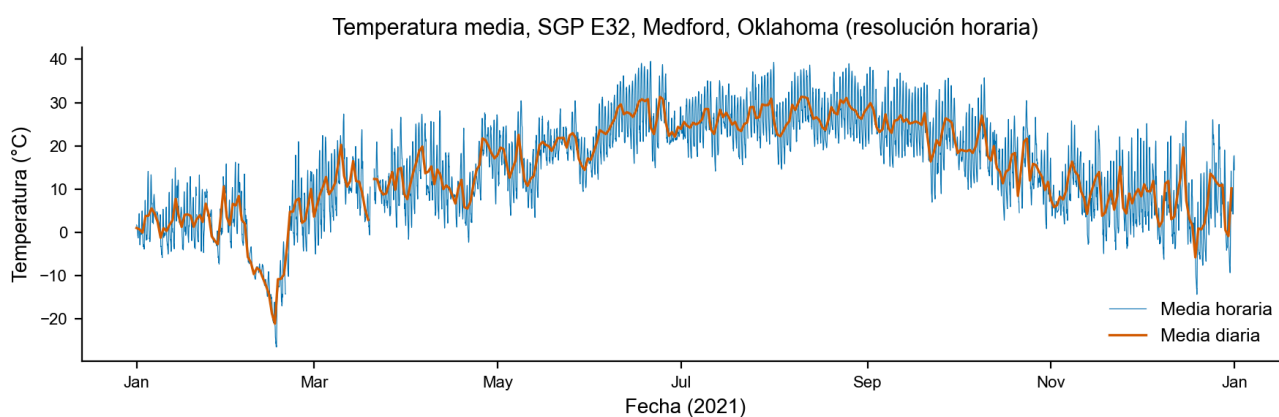


Figura 1. Serie temporal anual de temperatura: media horaria (azul) y media diaria (naranja). Se ve el ciclo estacional completo y el evento extremo de frío de febrero de 2021, la ola de frío histórica de Norteamérica, con un mínimo de $-27.1\text{ }^{\circ}\text{C}$.

Medias diarias de variables meteorológicas, sgpmetE32.b1 (2021)



Figura 2. Medias diarias de las cuatro variables principales durante 2021: (A) temperatura, (B) humedad relativa, (C) presión atmosférica y (D) velocidad del viento.

1.3 Detección de valores atípicos (outliers) y ruido

Apliqué dos métodos complementarios que vi en clase: el criterio **IQR** (valores fuera de $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$) y el **z-score** ($|z| > 3$).

Variable	Límites IQR	Outliers IQR	% IQR	Outliers $ z >3$	% z
Temperatura (°C)	[-18.6, 48.7]	1,760	0.34%	1,609	0.31%
Humedad relativa (%)	[-1.5, 143.3]	0	0%	0	0%
Presión atmosférica (kPa)	[96.1, 99.2]	10,432	2.00%	361	0.07%
Velocidad del viento (m/s)	[-2.1, 11.2]	12,769	2.45%	5,188	1.00%

Distribución y outliers (método IQR), datos crudos 2021

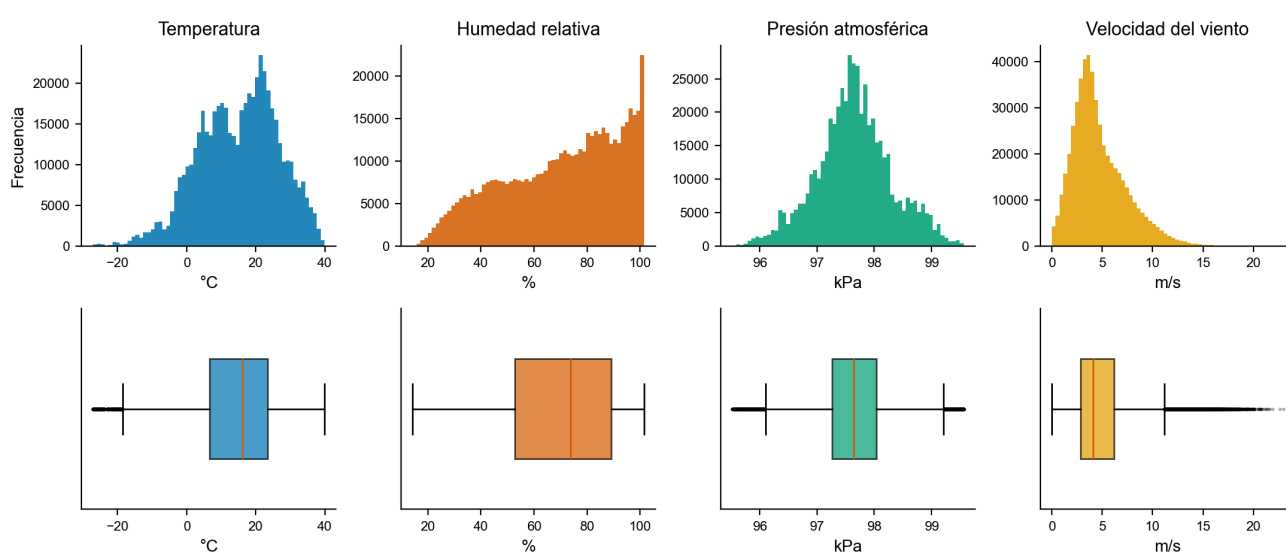


Figura 3. Histogramas (arriba) y diagramas de caja (abajo) de las variables principales. La temperatura es bimodal por la estacionalidad; el viento tiene asimetría positiva (1.04) con una cola larga de eventos intensos; la humedad relativa se acumula contra el techo de saturación, lo que se ve como una barra alta cerca del 100%.

Interpretación crítica. En datos meteorológicos un outlier estadístico no es lo mismo que un error. Los 1,760 outliers de temperatura son la **ola de frío histórica de febrero de 2021** (mínimo -27.1°C en Oklahoma). Es un evento real y con valor científico, así que no lo elimino. Aparte de los outliers, sí encontré dos cosas que conviene tratar:

- **22,463 registros (4.31%) con humedad relativa $> 100\%$.** No es un valor imposible: el sensor capacitivo HMP155 reporta hasta $\sim 104\%$ cerca de la saturación y el propio ARM fija `valid_max = 104` para esta variable. Es un comportamiento conocido del sensor que conviene acotar al techo físico de 100% para el análisis.
- **4,400 minutos sin registro**, repartidos sobre todo en marzo, julio y octubre (Fig. 4A).
- **5 saltos de temperatura mayores a $5^{\circ}\text{C}/\text{min}$** según un umbral exploratorio propio. Es un criterio más estricto que el del ARM, cuyo bit 4 de calidad usa `valid_delta = 20^{\circ}\text{C}/\text{min}`.

Calidad de los datos por mes (completitud temporal global: 99.16%)

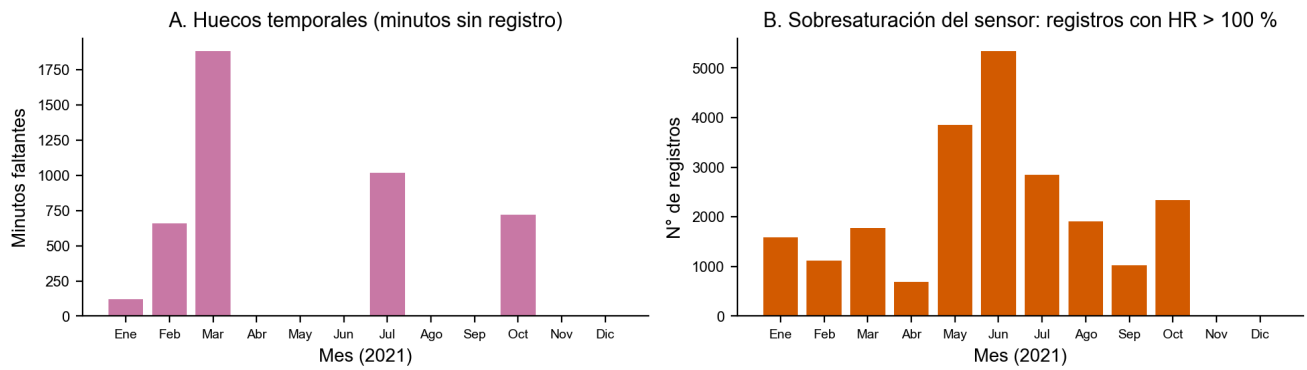


Figura 4. Calidad de los datos por mes: (A) huecos temporales, minutos sin registro, con máximo en marzo (1,880 min); (B) registros con HR > 100% por sobresaturación del sensor, más frecuentes en los meses húmedos de mayo a julio. Completitud temporal global: 99.16%.

Un detalle interesante: los **flags de calidad (QC)** del propio dataset apenas marcan 7 registros en todo el año. Al ser nivel **b1**, los datos ya pasaron el control de calidad automático del ARM. Esto refleja el concepto de **niveles de datos** que vi en clase y explica por qué el ruido restante (como la HR sobre 100%, que el rango válido del instrumento permite hasta 104%) necesita reglas físicas adicionales para tratarlo en el análisis.

1.4 Limpieza y transformación de datos

En la fase de limpieza del ciclo metodológico apliqué este pipeline (código en el Anexo A):

1. **Enmascarado por QC:** todo registro con `qc_* ≠ 0` pasa a `NaN`, siguiendo la regla del ARM de que un valor QC de cero significa que ninguna prueba falló.
2. **Acotado físico de la humedad:** los valores $> 100\%$ se recortan al techo de saturación (100%) con `clip(upper=100)`. Opté por acotar en vez de descartar para conservar el dato, ya que se trata de un comportamiento conocido del sensor y no de un error. Así no se pierde ningún registro de una variable válida.
3. **Imputación de faltantes:** interpolación lineal **temporal** limitada a tramos de hasta 60 registros consecutivos (≈ 60 minutos). Los huecos más largos los dejo como `NaN` para no inventar datos; prefiero cuidar la veracidad antes que forzar la completitud.
4. **Normalización:** generé versiones **Min-Max** ($[0,1]$) y **Z-score** ($\mu=0, \sigma=1$) de las variables. Hacen falta para algoritmos sensibles a la escala, porque las unidades originales son muy dispares (por ejemplo, la presión varía en un rango de ~ 4 kPa y la temperatura en unos 67°C).

Con el acotado de la humedad y la interpolación de tramos cortos, las ocho variables principales quedan con **100% de completitud** tras la limpieza. Hay que tener presente que la normalización Min-Max es sensible a extremos: el evento de febrero comprime el resto de los valores de temperatura, por lo que para modelos que lo requieran sería razonable un escalado robusto (mediana/IQR) o winsorizar antes de normalizar, manteniendo el evento en la serie cruda.

Acotado de la sobresaturación en humedad relativa (semana más afectada de 2021)

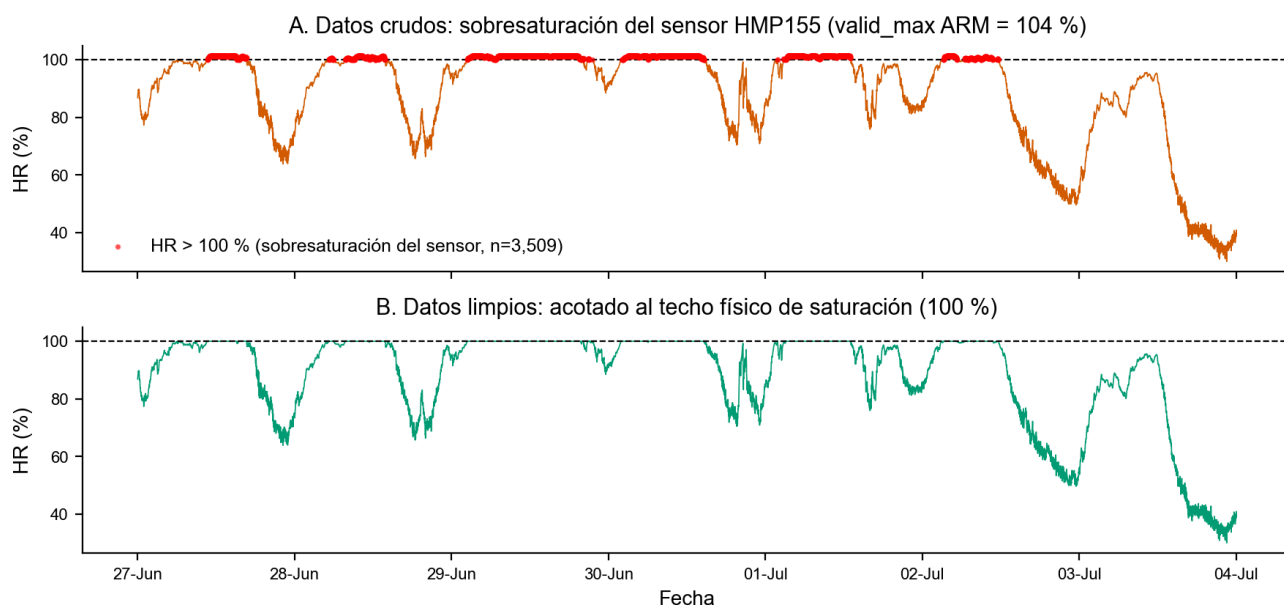


Figura 5. Acotado de la sobresaturación de la humedad en la semana más afectada (27 jun – 4 jul 2021, con 3,509 registros sobre 100%). (A) datos crudos con los valores sobre 100% marcados en rojo; (B) datos limpios tras acotar al techo físico de 100%.

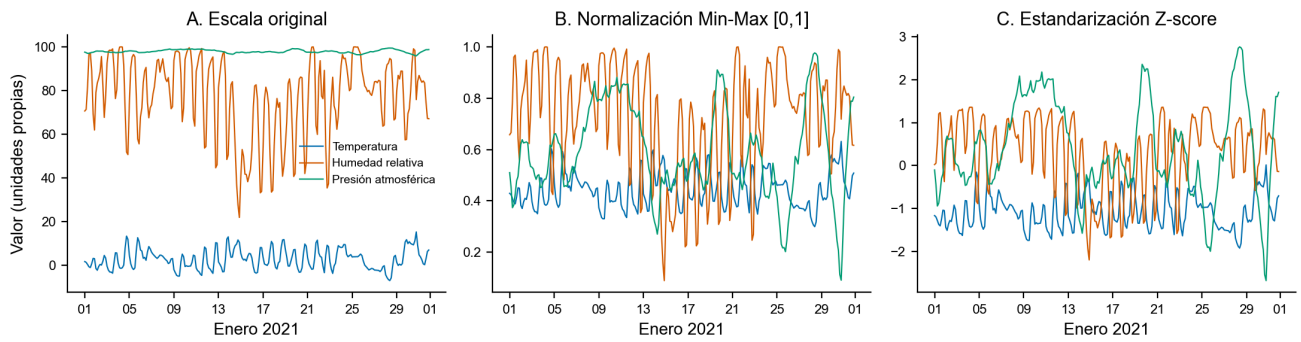


Figura 6. Efecto de la transformación (enero 2021): (A) escala original, donde la presión (~97 kPa) y la temperatura (~0 °C) no son comparables; (B) normalización Min-Max y (C) estandarización Z-score, que llevan todas las variables a escalas comparables. Verificación: min=0 y max=1 exactos en Min-Max; $\mu=0.00$ y $\sigma=1.00$ en Z-score.

Análisis de relaciones y patrones (datos limpios)

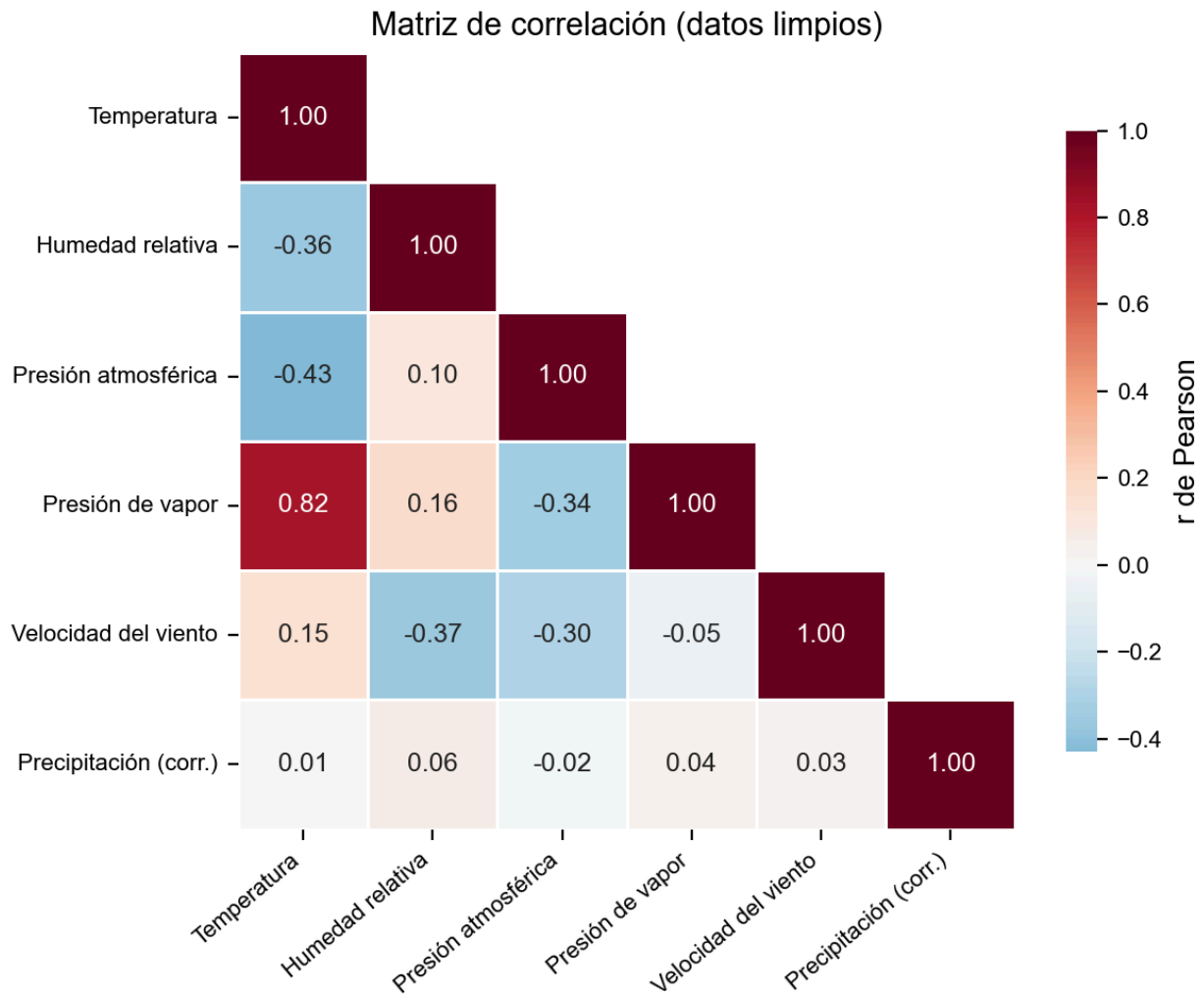


Figura 7. Matriz de correlación de Pearson. Resalta la correlación física fuerte entre temperatura y presión de vapor ($r = 0.82$, ley de Clausius-Clapeyron), la anticorrelación temperatura-presión ($r = -0.43$, sistemas de alta presión fríos) y temperatura-HR ($r = -0.36$, ciclo diario). La colinealidad entre temperatura y presión de vapor hay que tenerla en cuenta si se usan ambas como features de un modelo.

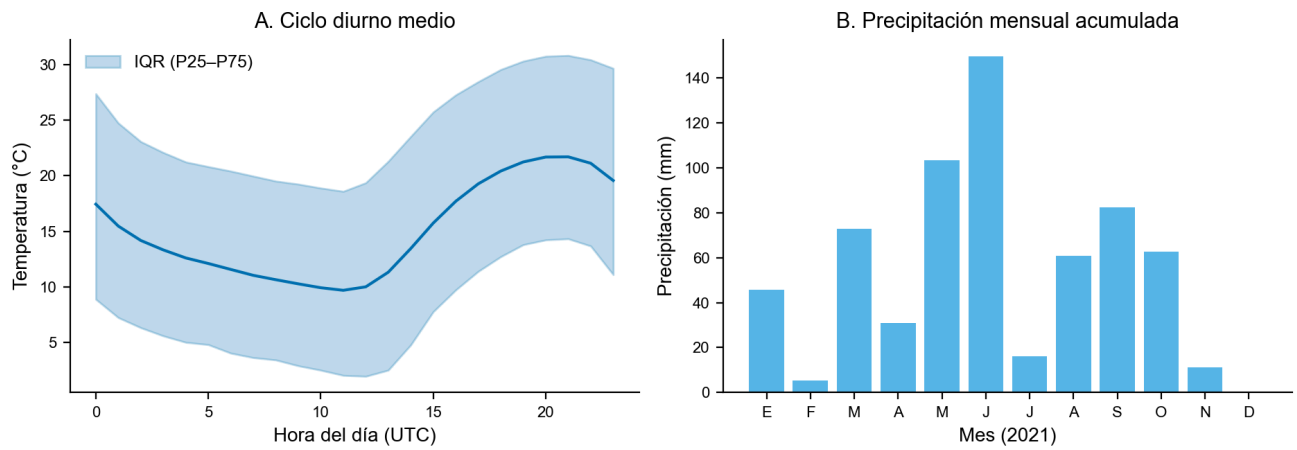


Figura 8. Patrones temporales: (A) ciclo diurno medio de temperatura con su banda intercuartílica (mínimo cerca de las 12 UTC, que es la madrugada local, y máximo cerca de las 21 UTC, la tarde local); (B) precipitación mensual acumulada (total anual 641.1 mm, concentrada en primavera y verano, un régimen típico de las Grandes Llanuras).

1.5 Desafíos del ciclo de vida de los datos en un contexto Big Data

Este dataset es una sola estación, un solo instrumento y un solo año, y ya tiene medio millón de registros y 70.9 MB. El programa ARM opera cientos de instrumentos en decenas de instalaciones desde 1996, con colecta continua que llega a petabytes. Ese es el escenario Big Data real que se discutió en clase. Reviso los desafíos por fase del ciclo de vida y los conecto con las **V's de Big Data**:

Fase del ciclo de vida	Desafío en contexto Big Data	V asociada
Generación / Colecta	Sensores que muestrean cada segundo o minuto, 24/7, en sitios remotos. Las fallas de energía e instrumento generan huecos, como los 4,400 minutos que faltan aquí.	Velocidad, Veracidad
Ingestión	Los procesos automáticos (<code>met_ingest</code>) tienen que operar sin intervención humana, versionados y auditables. El streaming continuo exige tolerancia a fallos.	Velocidad, Volumen
Almacenamiento	Décadas de datos crudos y procesados en varios niveles (00/a1/b1). Hacen falta formatos comprimidos y autodescriptivos (NetCDF, Parquet) y almacenamiento distribuido (data lake).	Volumen, Variedad
Control de calidad	No se pueden inspeccionar a mano 10^6 – 10^9 registros. El QC se automatiza con flags bit-packed y, aun así, deja pasar comportamiento del sensor (HR > 100%) que requiere validación física adicional.	Veracidad, Validez
Catalogación / Metadatos	Sin metadatos ricos y DOIs los datos no se encuentran ni se pueden usar a escala. Los principios FAIR son la respuesta de gobernanza (el portal Data Discovery del ARM).	Valor
Análisis / Procesamiento	Analizar muchas estaciones por décadas no cabe en una sola máquina. Se necesita procesamiento distribuido (Spark sobre clusters, arquitecturas Lambda para batch más streaming).	Volumen, Velocidad
Visualización / Compartición	Comunicar millones de puntos obliga a agregar de forma inteligente (el remuestreo horario y diario de las Figs. 1 y 2) y a dashboards que escalen.	Visualización, Valor
Preservación / Reúso	Que datos de 1996 sigan siendo legibles y reproducibles en 2026 depende de estándares abiertos, documentación de instrumentos y versionado del software de ingesta.	Valor, Veracidad

1.6 Conclusión

El dataset **sgpmetE32.b1 (2021)** es de alta calidad y muy útil. Tiene una completitud temporal del 99.16% sin timestamps duplicados, metadatos ricos y autodescriptivos conformes a los principios FAIR, control de calidad documentado con flags bit-packed, y captura bien tanto los ciclos físicos esperados (estacional, diurno, la correlación temperatura-presión de vapor $r = 0.82$) como eventos extremos reales, como la ola de frío de febrero de 2021 con -27.1 °C. Sus limitaciones son menores y se corrigen con el pipeline aplicado: 4,400 minutos de huecos, un 4.31% de registros con sobresaturación de humedad propia del sensor, y la necesidad de reglas físicas que complementen el QC automático. Tras la limpieza las variables principales quedan al 100% de completitud. Por todo esto se concluye que el dataset sirve para análisis climatológicos, para modelos de predicción meteorológica y como caso de estudio de cómo gestionar el ciclo de vida de datos científicos en un entorno Big Data.

2. Arquitectura Escalable de Big Data para Análisis de Datos

Diseñé una arquitectura escalable para gestionar y analizar datos meteorológicos como los del programa ARM, que sirve para cualquier flujo masivo de datos de sensores. El diseño sigue el modelo de capas que vi en clase (plataforma de almacenamiento y procesamiento distribuido, gobernanza FAIR y capa de aplicaciones) y el flujo **Ingestión** → **Almacenamiento** → **Procesamiento** → **Catalogación/Metadatos** → **Indexado/Consulta** → **Visualización/Serving**, con **Monitoreo y Seguridad** como capa transversal.

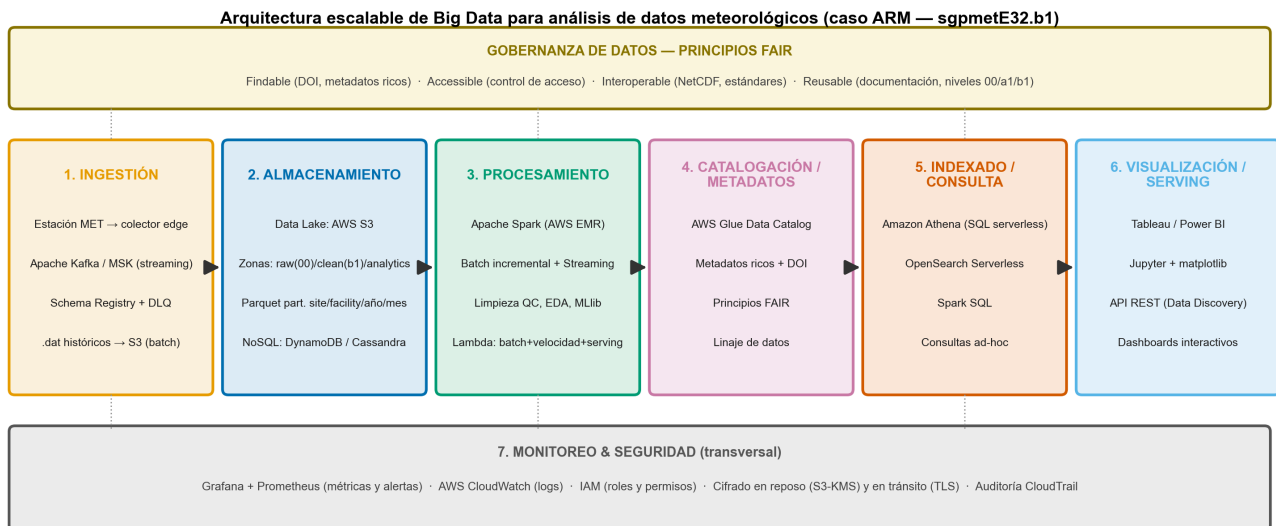


Figura 9. Diagrama de la arquitectura propuesta. Las bandas superior (gobernanza FAIR) e inferior (monitoreo y seguridad) son transversales a todo el pipeline.

2.1 Descripción paso a paso

Paso 1, Ingestión

En cada estación, un colector edge (o un agente productor) toma la salida del datalogger y la publica hacia el sistema central. Esto es más realista que conectar el datalogger directo a la red, sobre todo en sitios remotos. Para el flujo en **tiempo real** uso **Apache Kafka (o Amazon MSK)**, con un tópicos por datastream (por ejemplo `sgp.met.E32`), que desacopla productores de consumidores y aguanta picos de carga. Las cargas **batch** de archivos históricos `.dat` se suben directamente a S3, sin pasar por Kafka. En la ingesta valido el esquema contra un **Schema Registry** (versiona y verifica los contratos de datos) y los registros que fallan la validación van a una **cola de descarte (DLQ)** o **zona quarantine/** para reprocesarlos después. **Escalabilidad:** Kafka reparte el tópicos entre particiones, así que agregar estaciones solo pide más particiones.

Paso 2, Almacenamiento

Un **data lake en AWS S3** organizado en zonas que replican los niveles de datos del ARM: `raw/` (nivel 00, datos brutos inmutables), `clean/` (nivel b1, con QC) y `analytics/` (agregados). Guardo en **NetCDF** (interoperabilidad científica) y **Parquet** particionado por `site/facility/year/month`, una jerarquía que permite podar particiones en las consultas. Para accesos operativos de baja latencia (la última lectura de cada estación) uso una base **NoSQL: DynamoDB** (partition key = estación, sort key = timestamp) o **Cassandra** si se prefiere controlar el cluster. **Escalabilidad:** S3 crece sin límite práctico y el particionado evita escanear datos que no se necesitan.

Paso 3, Procesamiento

Apache Spark sobre AWS EMR (nodo primario con el driver y nodos core/task con executors, como se vio en clase). Monto una **arquitectura Lambda** con sus tres capas. La capa **batch** corre el pipeline de limpieza del EDA de forma **incremental**, procesando solo las particiones nuevas o afectadas (el día o mes recién aterrizado), y reserva el recálculo completo del histórico para cuando cambia la lógica o hace falta un backfill. La capa de **velocidad** (Spark Structured Streaming consumiendo de Kafka) calcula métricas casi en tiempo real y dispara alertas de eventos extremos. La capa de **serving** reconcilia las vistas batch e incremental para que cada consulta lea la combinación correcta. **MLlib** permite entrenar modelos distribuidos. Elijo Lambda sobre Kappa porque busco un batch reproducible sobre el histórico, asumiendo el costo de mantener dos rutas de cómputo. **Escalabilidad:** EMR agrega y quita nodos según la carga (scale-out horizontal).

Paso 4, Catalogación / Metadatos

AWS Glue Data Catalog registra de forma automática (con crawlers) los esquemas, las particiones y las estadísticas de cada tabla del data lake. Cada datastream recibe un **identificador persistente (DOI)**, documentación del instrumento y linaje de datos (qué versión de software produjo qué archivo). Esta capa concreta los principios **FAIR**: sin catálogo, un data lake de petabytes se vuelve un "data swamp" donde no se encuentra nada.

Paso 5, Indexado / Consulta

Amazon Athena permite consultas SQL serverless directamente sobre el Parquet del data lake usando el catálogo de Glue (se paga por datos escaneados, y el particionado del Paso 2 reduce ese costo). Para búsqueda de texto y metadatos y para exploración facetada, como el portal Data Discovery del ARM, uso **OpenSearch Serverless**. Los científicos de datos también consultan con **Spark SQL** desde notebooks. **Escalabilidad:** Athena es serverless; DynamoDB se usa en modo on-demand y OpenSearch Serverless escala solo.

Paso 6, Visualización / Serving

Tableau (o Power BI) conectado a Athena para dashboards interactivos; **JupyterHub** con matplotlib y seaborn para el análisis científico reproducible, como el EDA de la Parte 1; y una **API REST** que sirve datos a aplicaciones externas, igual que el portal del ARM. La clave de escalabilidad acá es servir **agregados precalculados** (medias horarias y diarias) en lugar de los 521 mil puntos por estación-año.

Paso 7, Monitoreo y Seguridad (transversal)

Monitoreo: Grafana y Prometheus muestran las métricas de salud del pipeline (lag de Kafka, duración de los jobs Spark, porcentaje de registros con flags QC por día; una caída de completitud como la de marzo de 2021 dispararía una alerta). CloudWatch centraliza los logs. **Seguridad:** IAM con roles de mínimo privilegio por capa, cifrado en reposo (S3 con KMS) y en tránsito (TLS), auditoría con CloudTrail y control de acceso por zona del data lake (la zona `raw/` es de solo escritura para la ingesta e inmutable para el resto).

2.2 Ejemplo práctico: el viaje de un dato

Sigo el recorrido de una lectura de temperatura de la estación SGP E32 del 15 de febrero de 2021, durante la ola de frío, con -27.1°C :

1. **00:01 UTC, Ingestión:** el datalogger entrega su promedio del último minuto al colector edge de la estación, que lo publica (`{"st":"sgpmetE32","t":"2021-02-15T00:01Z","temp":-27.1,"rh":78.2,...}`) al tópico Kafka `sgp.met.E32` tras validar el esquema.
2. **+2 s, Capa de velocidad:** Spark Structured Streaming detecta que -27.1°C cruza el umbral de evento extremo y, en pocos segundos, levanta una alerta en Grafana y escribe el último valor en DynamoDB (`PK=sgpmetE32, SK=2021-02-15T00:01Z`) para el dashboard en vivo.
3. **+1 h, Almacenamiento raw:** un consumidor de Kafka (Kafka Connect con S3 Sink) consolida el micro-batch horario y lo deja en `s3://datalake/raw/sgp/met/E32/2021/02/15/` (nivel 00, inmutable).
4. **+1 día, Procesamiento batch:** el job Spark nocturno en EMR procesa solo la partición nueva, aplica el pipeline de limpieza (flags QC, acotado de HR a 100%, interpolación de tramos cortos), verifica que -27.1°C es un evento real (es consistente en horas contiguas y en las estaciones vecinas E13 y E15, no un artefacto del sensor) y escribe el Parquet limpio en `clean/` (nivel b1).
5. **+1 día, Catalogación:** el crawler de Glue registra la nueva partición. El datastream conserva su DOI y el linaje guarda qué versión del job la produjo.
6. **Consulta:** una climatóloga corre en Athena, filtrando por las columnas de partición:

```
SELECT date_trunc('day', time) AS dia, min(temp_mean) AS t_min
FROM clean.met
WHERE site='sgp' AND facility='E32' AND year=2021 AND month=2
GROUP BY 1 ORDER BY 2 LIMIT 5;
```

y obtiene en segundos los días más fríos del evento, escaneando solo la partición de febrero.

7. **Visualización:** el dashboard de Tableau muestra la anomalía de febrero frente a la climatología, y el informe (como este documento) se arma desde JupyterHub con los agregados.
8. **Transversal:** cada paso emitió métricas a Prometheus y logs auditables a CloudWatch; todos los accesos pasaron por roles IAM y los datos viajaron cifrados.

Por qué el diseño escala. Cada componente crece en horizontal: particiones de Kafka, S3 sin límite, nodos EMR elásticos, Athena serverless, DynamoDB on-demand. Pasar de 1 estación (0.5 M registros al año) a las ~30 instalaciones del SGP con decenas de instrumentos (más de 10^9 registros al año) no cambia la arquitectura: solo se agregan particiones y nodos. El procesamiento batch incremental evita reprocesar todo el histórico a diario, lo que mantiene el costo acotado. Y separar almacenamiento de cómputo (S3 frente a EMR) deja apagar los clusters cuando no se usan.

Anexo A. Reproducibilidad: código esencial del EDA

Entorno: Python 3.14, xarray 2026.4, pandas, numpy, matplotlib, seaborn. Script completo: `src/eda_arm_sgpmet.py`.

```
import xarray as xr, pandas as pd, numpy as np

# 1) Carga del NetCDF y conversion a DataFrame temporal
ds = xr.open_dataset("ARM_sgpmetE32.b1.2021.nc")          # 521,200 x 31 data_vars
df = ds[vars_fisicas + vars_qc].to_dataframe()

# 2) Integridad y completitud temporal (esperados: 365*1440 = 525,600 min)
dup = df.index.duplicated().sum()                        # 0 duplicados
completitud = 100 * len(df) / 525600                    # 99.16 %

# 3) Outliers: IQR y z-score
q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1
out_iqr = (s < q1 - 1.5*iqr) | (s > q3 + 1.5*iqr)
out_z = np.abs((s - s.mean()) / s.std()) > 3

# 4) Limpieza: flags QC -> NaN; acotado fisico de HR; imputacion temporal
clean = df[vars_fisicas].mask(df[vars_qc].ne(0).values) # QC != 0 -> NaN
clean['rh_mean'] = clean['rh_mean'].clip(upper=100)      # techo de saturacion
clean = clean.interpolate(method='time', limit=60)      # tramos <= 60 min

# 5) Transformacion: normalizacion
minmax = (clean - clean.min()) / (clean.max() - clean.min())
zscore = (clean - clean.mean()) / clean.std()
```

Referencias

- ARM, Atmospheric Radiation Measurement user facility. *Surface Meteorological Instrumentation (MET)*, datastream sgpmetE32.b1. ARM Data Center. <https://adc.arm.gov/discovery/#/results/s::sgpmetE32>
- Encinas Quille, R. V. (2026). *DataLifecyclePy*, repositorio del curso. <https://github.com/encinasquille/DataLifecyclePy>
- Wilkinson, M. D. et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018.
- Materiales del curso: Clase 2, Ciclo de vida de datos y Hadoop; Clase 3, Procesamiento distribuido; Clase 4, AWS EMR con Apache Spark; Clase 6, Scalable Data Visualization in Big Data Architectures; Clases 7 y 8, NoSQL (Cassandra, DynamoDB).
- Zaharia, M. et al. (2016). Apache Spark: a unified engine for big data processing. *Communications of the ACM*, 59(11).